

Dev-Hub is a webapp that is going to solve and help memory tracking

I. THE CORE PHILOSOPHY (THE "LIVING ROOM")

The NodeLink Hub is not a project manager, a to-do list, or an IDE. It is a **Digital Dev-Twin**. Its sole purpose is to prevent *Context Decay*. It bridges the gap between the rigid, dry history of Git commits and the fluid, emotional, and raw logic of the developer's brain. It acts as a memory anchor, ensuring that returning to a project after days or months feels like walking back into your own living room, exactly as you left it.

II. THE DUAL-LAYER UI ARCHITECTURE

The interface is strictly divided into two synchronized views that never overlap but constantly talk to each other.

- **Layer 1: The "Weird Stuff" Feed (The Human Layer)**
 - A low-friction, chat-like interface where the developer logs raw thoughts, frustrations, and momentum cues.
 - *Example input:* "The Scaffold top bar is fighting the system bars padding again in `RadarScreen` . I'm disabling the padding."
 - **Purpose:** Captures the *Why* and the *Vibe* behind the code.
 - **Layer 2: The Tactical Brief (The Data Layer)**
 - A visual dashboard pulling live data via GitHub Webhooks.
 - Shows a "Stability Heatmap" of which files are currently being modified the most.
 - Displays the "Feature Wire Map"—a visual node-graph generated by the AI showing how the developer's raw ideas connect to the actual codebase.
-

III. THE 5 VITAL ORGANS (CORE FEATURES)

1. The Intention Gate & Session Tracker

- Developers do not just start coding; they "Initiate a Mission" by declaring an intent (e.g., "Fixing the ghost padding").
- A non-intrusive "Lock-In" Pomodoro runs silently in the background. If the developer goes silent for too long, the Hub gently nudges them using their

own stated intent to break the mental block.

2. Semantic Commit Intelligence (The GitHub Hook)

- The Hub listens to GitHub for every `git push`.
- It uses AI to read the code diffs and automatically links the code changes to the developer's latest logs in the Weird Stuff Feed.
- It auto-generates a summary explaining *what* changed and *why* it changed, saving the developer from writing massive documentation.

3. The End-of-Session Handshake (Anti-Regression)

- Before the developer logs off, the Hub audits the session.
- If the developer logged that a bug was fixed but no commit was pushed to GitHub, the Hub intercepts: *"Yo, you mentioned the Radar sweep is fixed, but it's not in the biography (Git). Commit now so we don't lose the context."*

4. Smart Project Freezing & Resuming

- When a developer switches to a new project, the Hub "packages" the current project's consciousness.
- Upon returning, the Hub delivers a 3-sentence briefing of exactly where the developer left off, the files that were hot, and the last unresolved thought in the Weird Stuff feed.

5. The Feature Brainstormer (The Co-Architect)

- An interactive chat state where the developer pitches ideas.
- The Hub logs these ideas, validates them against the current codebase, and compiles them into a living "Feature Wire Map" so massive ideas are never forgotten.

IV. THE TECHNICAL STACK (THE ENGINE)

To keep the Hub lightweight, decoupled from the OS, and highly intelligent:

- **Front-End (The UI):** `Next.js` (React) styled with `Tailwind CSS`. It needs to be fast, dark-mode native, and highly responsive.
- **Back-End (The Brain):** `FastAPI` (Python) to handle the complex AI routing, prompt management, and GitHub webhook ingestion.
- **Database (The Memory):** `Supabase` (PostgreSQL with `pgvector`). Essential for storing the developer's text logs and performing "similarity searches" so the AI can remember past solutions to current bugs.

- **Integrations:**
 - `GitHub API` (for commit and file tracking).
 - `Gemini Pro API` (for analyzing code diffs and understanding the developer's "Weird Stuff" logs).
 - **Local Handoff:** Protocol handlers (e.g., `vscode://` or `studio://`) embedded in the web UI so clicking a file in the Hub instantly opens the developer's local IDE to that exact file.
-

V. THE DEVELOPER USER FLOW

1. **Enter:** Developer opens the webapp. Hub says, *"Welcome back. Last time, we left the RSSI math half-baked in the ViewModel."*
2. **Declare Intent:** Developer types: *"Today we are finishing the distance formula and testing the radar."*
3. **Execute:** Developer codes locally. They hit a bug and type into the Hub: *"Math is returning negative values, something is wrong with the signal strength variable."*
4. **Resolve & Push:** Developer fixes the code and pushes to GitHub.
5. **Sync:** The Hub sees the push, reads the diff, pairs it with the negative value log, and marks the session intent as "Resolved."
6. **Exit:** Developer closes the Hub. The Hub freezes the state for next time.